Python

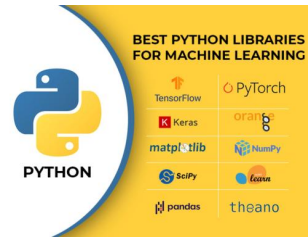
Simão Paredes
sparedes@isec.pt

Outline

- Why Python?
- Essential Libraries and Tools
  - scikit-learn
  - NumPy
  - SciPy
  - matplotlib
  - seaborn
  - pandas
- Jupyter Notebook
  - Colab (https://colab.research.google.com)
  - Anaconda
- Python Language

Why Python?

- Well adapted to Data Science Application
  - Simple syntax, a very readable language with clear non-verbose syntax
  - Efficient code
  - It has libraries for:
    - machine learning
    - deep learning
    - data visualization
    - statistics, …
  - Freeware
    - Even for building commercial software



Python Libraries

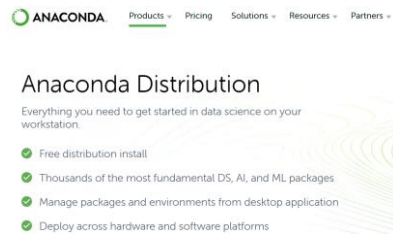
- Python Ecosystem for Machine Learning



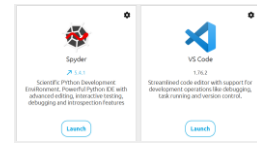
https://devopedia.org/machine-learning-in-python
https://www.simplilearn.com/keras-vs-tensorflow-vs-pytorch-article

Python

- Local Runtime







https://www.anaconda.com/products/individual

- Hosted Runtime
  - colab
    - Zero configuration required
    - Free access to GPUs
      - https://colab.research.google.com/#scrollTo=GJBs_flRovLc



Essential Libraries and Tools

- scikit-learn



- Most prominent Python library for Machine Learning applications
- Contains a large number of state of the art machine learning algorithms
- Comprehensive documentation
- Continuously developed and improved
- Very active user community

Essential Libraries and Tools

- scikit-learn depends on two Python packages:
  - NumPy
  - SciPy

  - NumPy
    - A fundamental package, as it contains functionality for **multidimensional arrays** as well as high level mathematical functions

```
import numpy as np
```



  - SciPy
    - Collection of functions for **scientific computing**, such as advanced linear algebra, mathematical functions, optimization, statistical distributions, …

```
from scipy import sparse
```

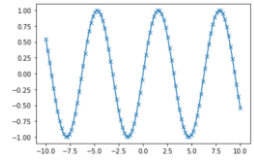


Essential Libraries and Tools

- matplotlib
  - provides functions for visualizations, like charts, histograms, scatters, …

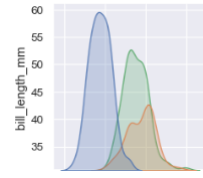
```
import numpy as np
import matplotlib.pyplot as plt

x=np.linspace(-10,10,100)
y=np.sin(x)
plt.plot(x,y,marker='x')
```



matplotlib

- seaborn
  - enhanced visualizations



seaborn

Essential Libraries and Tools

- pandas



  - Provides several methods to operate and modify a DataFrame
    - Two-dimensional, size-mutable, potentially heterogeneous tabular data
  - Very useful to gather data from several file formats
    - e.g. *.csv , *.xlsx , …

```
import pandas as pd
data={'Name': ['John', 'Anna', 'Peter', 'Linda'],
      'Location': ['NY', 'Paris', 'London', 'Berlin'],
       'Age': [34, 23, 38, 54]}
data_pandas=pd.DataFrame(data)
display(data_pandas)
```

	Name	Location	Age
0	John	NY	34
1	Anna	Paris	23
2	Peter	London	38
3	Linda	Berlin	54

pandas

Standard flow

pandas

Load, clean, and manipulate input data

scikit_learn

Pandas DataFrame is converted into a NumPy array

This array is passed into some function

scikit_learn needs SciPy functions for working properly.

Main Concepts

Python



Python - Basics

- Interpreted language
- Multi platform (Windows, MacOS, Linux, …)
- Case sensitive
- # comment symbol (single line)
  - Multiline comment, based on a multiline string (**‘’’**) that is ignored if it is not assigned to a variable

Python - Fundamentals

- Basic data types
- Containers
  - Strings
  - Lists
  - Dictionaries
  - Tuples
  - Sets
- Functions
- Classes

Basic Data Types

Python

Basic Data Types

- Python is a dynamically typed language:
  - The variable´s type is not declared
  - The type of the variable is not predefined until the code is run.

integer | *float* | *Complex numbers* | *Booleans*

```
num3 = 2 + 6j
print(num3)
print('Dtype of num3 is',type(num3))
```

```
(2+6j)
Dtype of num3 is <class 'complex'>
```


Containers

Python

Data Types

Containers

Strings | *Lists* | *Dictionaries* | *Tuples* | *Sets*

Python – Containers

- Strings
  - Different string's syntaxes
    - Single or double quotes may be applied

```
1 s='single quotes'
2 d="double quotes"
3 m='''multiple
4 lines'''
5
6 print(s)
7 print(d)
8 print(m)
```

```
single quotes
double quotes
multiple
lines
```

  - Index starts at 0
  - Negative indices (count from the end)

```
s="firstString"

print(s[0])
print(s[1])

print(s[-1])
```

```
f
i
g
```

Python – Containers

- Strings
  - slicing
    - [start:stop]
    - [start:stop:step]
      - The **upper limit is not included!**

```
s="firstString"
print(s[2:5])     #the upper limit is excluded
print(s[2:8:2])   #  a[start:stop:step]
print(s[::2])     # every 2 characters from beggining to end
```

  - An **immutable object!**
    - It is not possible to modify the original string

Python – Containers

- List
  - An ordered collection of objects/values that may have **different types** defined with [ ] and elements separated with comma
    - Indexing starts at 0

```
colors=['red', 'blue', 4, 'black', 5]
print(colors[1:4])        # end position is excluded
print(colors[:2])         # end position is excluded
print(colors[2:])         # from the start position until the end
```

    - Slicing similar to strings
      - [start: stop] contains the elements with index i such that $\mathbf{start} \leq \mathbf{i} < \mathbf{stop}$, thus it has $(stop - start)$ elements
    - **Important:** Python list ≠ Numpy array

      Numpy array **is much more efficient** for operations with numerical data.

Python – Containers

- List
  - Lists are **mutable** objects

```
1 colors=['red',5,'green','blue']
2 colors[1]='orange'
3 print(colors)
```

  - Python lists have a large collection of methods
    - append()

```
1 colors.append('black')
2 print(colors)
```

```
['red', 'orange', 'green', 'blue', 'black']
```

    - pop()

```
1 colors.pop()
2 print(colors)
```

```
['red', 'orange', 'green', 'blue']
```

    - extend()

```
1 colors.extend(['pink','olive'])
2 print(colors)
```

```
['red', 'orange', 'green', 'blue', 'pink', 'olive']
```

    - …

Python – Containers

- Lists
  - **+** operator
    - Concatenation

```
1 colorbase=['red','blue']
2 colorlight=['lightred','lightblue']
3 colors=colorbase+colorlight
4 print(colors)
```

```
['red', 'blue', 'lightred', 'lightblue']
```

  - * operator
    - Repetition

```
1 colors=2 * colorlight
2
3 print(colors)
```

```
['lightred', 'lightblue', 'lightred', 'lightblue']
```

  - …

Python – Containers

- Dictionaries
  - maps **keys to values**

```
1 tel={'Mary':93458589, 'John':967564537 }
2 print(tel)
```

```
{'Mary': 93458589, 'John': 967564537}
```

  - Access to keys and values
    - keys()
    - values()

```
1 print(tel.keys())
```

```
dict_keys(['Mary', 'John'])
```

```
1 print(tel.values())
```

```
dict_values([93458589, 967564537])
```

  - Very useful to create pandas DataFrames, change columns names, …
  - New elements can be added to a dictionary

```
1 tel['Peter']=912345857
2 print(tel)
```

```
{'Mary': 93458589, 'John': 967564537, 'Peter': 912345857}
```


Python – Containers

- Dictionaries
  - Remove a key from a dictionary

```
1 tel.pop('Mary')
2 print(tel)
```

```
{'John': 967564537, 'Peter': 912345857}
```

  - a dictionary can have keys with different types

```
1 d={'a':1, 'b':2, 3:4}
2 print(d)
```

```
{'a': 1, 'b': 2, 3: 4}
```

```
1 print(d.keys())
```

```
dict_keys(['a', 'b', 3])
```

  - methods
    - help(dict)

Python – Containers

- Tuples
  - A tuple is an **immutable** ordered list of values.
  - The elements of a tuple are written between parenthesis or just separated by commas

```
1 t=(1,2,3,4,5)
2 print(t)
```

```
(1, 2, 3, 4, 5)
```

```
1 t=1,2,3,4,5,6,7
2 print(t)
```

```
(1, 2, 3, 4, 5, 6, 7)
```

  - A tuple allows indexing and slicing

```
1 t=(1,2,3,4,5)
2 t1=t[:2]
3 print(t1)
```

```
(1, 2)
```


Python – Containers

- Sets
  - A set is an **unordered, unindexed** collection of **distinct** elements
    - A set can contain different data types:

```
1 colors={'red','orange','blue','lightgreen','green', 'white'}
2 print(colors)
```

```
{'red', 'lightgreen', 'white', 'orange', 'blue', 'green'}
```

  - There are some methods specific for working with sets
    - add an element

```
1 colors.add('darkorange')
2 print(colors)
```

```
{'red', 'lightgreen', 'white', 'orange', 'darkorange', 'blue', 'green'}
```

    - remove an element

```
1 colors.remove('darkorange')
2 print(colors)
```

```
{'red', 'lightgreen', 'white', 'orange', 'blue', 'green'}
```



Exercise 1

Exercise 1

- Define a dictionary that represents the following table:

Expenses	Price
conference	500
hotel	200
flight	300

- Calculate the total amount of expenses.
  - sum()



End - Exercise 1

Operators

Python

Python – Operators

- Python:
  - does not have the operators ++ and --
  - basic arithmetical operators
  - ** exponentiation
  - Logical operators for Booleans (and, or, not, logical != (xor))

```
t = True
f = False

print(type(t)) # Prints "<class 'bool'>"
print(t and f) # Logical AND;
print(t or f)  # Logical OR;
print(not t)   # Logical NOT;
print(t != f)  # Logical XOR;
```

```
<class 'bool'>
False
True
False
True
```


Control Flow

Python

Python – Control Flow

- Conditional Expressions
  - if / elif / else
    - IMPORTANT: Blocks are delimited by **indentation**

```
a=2

if a==1:
    print(a)
elif a==2:
    print(a**2)
else:
    print('higher than 2')
```

```
4
```

  - Evaluates to false
    - Any number equal to 0
    - An empty container
    - false, None
  - Evaluates to True
    - everything else

```
a=[1,2,3,4]
print(1 in a)
```

```
True
```

```
print(20 in a)
```

```
False
```


Python – Control Flow

- Loops
  - while / break / continue

```
counter = 1

while counter <= 5:
    print("Counter is:", counter)
    counter += 1
```

```
Counter is: 1
Counter is: 2
Counter is: 3
Counter is: 4
Counter is: 5
```

  - for … in … range:
    - Built-in function **range()** creates a set of integers from 0 to N-1

```
for i in range(4):
    print(i)
```

```
0
1
2
3
```

Python – Control Flow

- Loops
  - In Python is possible to operate over any sequence such as strings, list, dictionary keys, …
  - for … in … :
    - Iteration over a string

```
vowels="aeiou"

for z in 'powerful':
    if z in vowels:
        print(z)
```

```
o
e
u
```

  - for … in … :
    - iterates through a list

```
animals=['cat','dog','bird','fish']
for i in animals:
   print(i)
```

```
cat
dog
bird
fish
```


Python – Control Flow

- Loops
  - for … in enumerate (…)
    - built-in function **enumerate()**
    - keeps the track (index) of the item number

```
animal=['cat','fish','dog']
for i,j in enumerate(animal):
    print('position:',i, 'animal:', j)
```

```
position: 0 animal: cat
position: 1 animal: fish
position: 2 animal: dog
```

  - for … in dictionary.items (…)
    - Loop over a dictionary based on **items()**
    - allows to access separately to key and value

```
d={'K1':'V1', 'K2':4, 5:'V3'}
for i,j in d.items():
  print('key:',i, 'value:', j)
```

```
key: K1 value: V1
key: K2 value: 4
key: 5 value: V3
```

Python – Control Flow

- List Comprehensions
  - Creation of a list with a more condensed and self-explaining syntax
  - Alternative to “classical” loops

```
b=[i**2 for i in range(6)]
print(b)
```

```
[0, 1, 4, 9, 16, 25]
```

** power operator

- Dictionary Comprehensions
  - Comprehensions can also be applied to dictionaries
    - e.g. create a dictionary where the key is a number and the value the respective square considering the list [1,2,3,4,5]

```
a=[1,2,3,4,5]
d= {i:i**2 for i in a}
print(d)
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```




Exercise 2

Exercise 2

- Ask the user for a word and count how many vowels it has.
  - input() : data input by the user



End - Exercise 2



Exercise 3

Exercise 3

- Create two lists, one with even numbers and another with odd numbers, based on a string of digits entered by the user (e.g. 1234).



End - Exercise 3



Exercise 4

Exercise 4

- Create a filter to identify names with less than 4 letters. The user must enter 5 names



End - Exercise 4



Exercise 5

Exercise 5

- Based on List Comprehensions calculate the squares only for the even numbers of nums
  - nums = [0,1,2,3,4,5,6,7] ?

Exercise 5

- Create a dictionary comprehension based on num=[0,1,2,3,4,5,6], such that:
  - the key is a number and the value the respective square but only if the original number is higher than 2

Exercise 5

- Consider a list made up of sublists. Using a list comprehension, calculate the sum of the values of the inner lists.



End - Exercise 5

Functions

Python

Python – Functions

- Functions
  - are defined through ***def*** followed by the function name
  - have to be indented similarly to other control-flow blocks

```
def disk_area(radius):
    return 3.14*radius*radius

disk_area(2)
```

```
12.56
```

  - the arguments of the function are given between parenthesis followed by a colon
    - they can be optional
      - allow to specify default values
      - are very useful for defining functions with a variable number of arguments, especially when default values are to be used in the majority of calls to that function

```
1  dfx=20
2  def fcn(x=dfx):
3      return x**2
4
5  b=fcn(4)
6  c=fcn()
7
8  print(b, '   ', c)
9
```

```
16     400
```


Python – Functions

- Parameters
  - If the value passed in a function is **immutable** (int, float, bool, string, tuple)
    - The function does not modify the caller variable.
  - If the value passed in a function is **mutable** (list, dictionary, set and user-defined classes)
    - The function can modify the caller's variable

```
def tryModify(x,y):
  x=2
  y=y.append(35)
  return x,y

a=20
b=[10,30]

tryModify(a,b)

print('int:', a)
print('list:', b)
```

```
int: 20
list: [10, 30, 35]
```

Reusing Code (modules)

Python

Python

- The extension of python files is *.py
  - A **module** is a file *.py with a set of functions, that can be imported and used by other file (code reuse)
    - The module must imported
    - The name of the module must be specified before the name of the function

D: > codePython > simpleCalc.py > ...

```
1  def calculate_square(i):
2      return i**2
3
4  def calculate_cube(i):
5      return i**3
6
```

```
import simpleCalc

simpleCalc.calculate_square(5)
```

```
25
```

    - Functions are objects that can also be directly imported into the main namespace

```
from simpleCalc import calculate_cube
calculate_cube(4)
```

```
64
```

      - In this case, the module's name (i.e. simpleCalc) is omitted.
  - A **package** is a directory that contains several modules

Object Oriented Programming (OOP)

Python

Python - OOP

- A class is a blueprint of objects
  - **__init__()** is a reserved method in Python classes.
    - It is the constructor method
      - It is called automatically every time the class is being used to create a new object.
      - When an object is created from the class, it allows initialization the of the class's attributes.
  - **self**
    - represents the instance of the class.
      - By using the **self** it is possible to access variables/attributes that belong to the class.

```
1  class Student(object):
2      def __init__(self,a,b,c):
3          self.d=a
4          self.e=b
5          self.f=c
6      def setAge(self):
7          print('Age:',self.f)
8      def setGender(self):
9          print('Gender:',self.e)
10
11 newStudent=Student('john','male',46)
12 newStudent.setAge()
13 newStudent.setGender()
```

```
Age: 46
Gender: male
```

object means that class Student inherits from object i.e. gets the core functionality of all objects.

Python - OOP

- **setAge()**; **setGender()** are methods from the Student class
- **newStudent** is an instance (object) of class Student

```
1  class Student(object):
2      def __init__(self,a,b,c):
3          self.d=a
4          self.e=b
5          self.f=c
6      def setAge(self):
7          print('Age:',self.f)
8      def setGender(self):
9          print('Gender:',self.e)
10
11 newStudent=Student('john','male',46)
12 newStudent.setAge()
13 newStudent.setGender()
```

```
Age: 46
Gender: male
```

The object means that class Student inherits from object i.e. gets the core functionality of all objects.

Python - OOP

- Inerithance in Python
  - **MasterStudent** is defined with the same attributes and methods than class **Student** but with an additional attribute **internship**

```
1  class Masterstudent(Student):
2         internship='from march to july'
3
4  newMasterstudent=Masterstudent('John',30,'Male')
5
6  newMasterstudent.setAge()
7  newMasterstudent.setGender()
8
9  print('Internship : ', newMasterstudent.internship)
```

```
age : 30
Gender : Male
Internship :  from march to july
```

Class Masterstudent inherits from Student i.e. gets the core functionality of the Student class.

  - **newMasterStudent is** an instance (object) of class **MasterStudent** that inherited attributes/methods from class **Student**



Exercise 6

Exercise 6

- Define a class with two attributes (name, age) and a method printName that implements a condition dependent of an Boolean parameter.
  - If that argument is false the name is printed in uppercase otherwise in lowercase.
    - Output example: " Name: MARY ; Age: 40 "



End - Exercise 6